PrintOnline.et v2.0 — Master Architecture Analysis & Strategy Blueprint

> **Document Version:** 2.0.0-DRAFT
> **Date:** 2026-02-26
> **Status:** PRE-DECISION ANALYSIS
> **Author:** Antigravity AI + Kidus (Architecture Lead)
> **Scope:** Full-stack E-Commerce Platform for Pana Promotion

---

Table of Contents

1. Executive Summary
2. Current State Assessment (v1.x)
3. Backend Strategy Analysis: Standalone Supabase vs ERP-Embedded CMS
4. Recommended Architecture: Supabase-First Standalone
5. Schema-First Domain Model
6. Feature-by-Feature Implementation Analysis
7. Authentication Strategy: better-auth + Supabase
8. Frontend Architecture Modernization
9. MVP-Today Scope vs Full v2.0 Roadmap
10. Risk Assessment & Pitfalls
11. Follow-Up Questions for Decision
12. Technology Stack Summary

---

1. Executive Summary

1.1 The Mission

Transform PrintOnline.et from a **static demo frontend** (hardcoded product data, localStorage cart, no user accounts) into a **production-grade e-commerce platform** backed by a real database, real authentication, and a minimalist CMS for managing products, orders, and customers.

1.2 Critical Constraints

Constraint	Impact
Push MVP today	Scope must be ruthlessly prioritized; schema + backend + data migration first
Client-provided product catalog (PDF)	Product data must be extracted, structured, and seeded into DB
Pana ERP integration deferred	ERP module as CMS was planned but business workflow misalignment forces standalone

Constraint	Impact
Ethiopia-first, international later	Currency (ETB), TIN fields, local payment methods, Ethio Telecom email
Solo developer velocity	Architecture must maximize code reuse and minimize boilerplate

1.3 Two Strategic Options Analyzed

Option	Architecture	Pros	Cons
A: Standalone Supabase	Supabase PostgreSQL + Storage + Auth (or better-auth)	Full control, fast to ship, no ERP dependency	Separate system to maintain, no ERP integration
B: ERP-Embedded CMS	Add "Web App" module in pana-erp repo, manage products/orders/customers via Frappe API	Single codebase, factory patterns reuse, CMS matches ERP UI	ERP workflow misalignment, heavier deployment, blocked if ERP isn't ready

> [!IMPORTANT] **My Recommendation: Option A (Standalone Supabase) for the MVP push, with a clean integration layer designed from day 1 so Option B can be layered in as the ERP stabilizes.** Detailed reasoning in Section 3.

---

2. Current State Assessment (v1.x)

2.1 What Exists Today

Layer	Current State	Assessment
Framework	Next.js 16 (App Router), React 19, TypeScript	☑ Solid foundation, modern stack
Styling	Tailwind CSS v4, shadcn/ui (new-york), Framer Motion	☑ Premium, well-themed with Pana branding
Product Data	Hardcoded in [lib/products.ts](file:///c:/Users/kidus/Documents/Projects/printonline-et/lib/products.ts) (~25 items)	✕ Must migrate to database
Product Types	[types/product.ts](file:///c:/Users/kidus/Documents/Projects/printonline-et/types/product.ts) + duplicate [Product](file:///c:/Users/kidus/Documents/Projects/printonline-et/lib/products.ts#2-24) interface in [lib/products.ts](file:///c:/Users/kidus/Documents/Projects/printonline-et/lib/products.ts)	⚠ Type drift issue, needs unification

Layer	Current State	Assessment
Cart	[context/CartContext.tsx] (file:///c:/Users/kidus/Documents/Projects/printonline-et/context/CartContext.tsx) with localStorage	⚠ Functional but needs DB persistence for auth users
Auth	None (better-auth is installed but not configured)	❌ Must implement
Routing	Static category pages + `products/[slug]` dynamic route	⚠ Needs catch-all modernization
Product Forms	Schema-driven system ([ProductFormSchemas.ts] (file:///c:/Users/kidus/Documents/Projects/printonline-et/components/product/ProductFormSchemas.ts), [ProductFormTypes.ts] (file:///c:/Users/kidus/Documents/Projects/printonline-et/components/product/ProductFormTypes.ts))	☑ Excellent pattern, needs data-driven options
Checkout Flow	`/checkout` → `/order-summary` → `/order-confirmation`	⚠ Functional UI, no backend persistence
Email	Planned (Nodemailer + Ethio Telecom SMTP)	📝 Workflow documented, not implemented
Search	Search icon in header, no functionality	❌ Must implement
Dark Mode	`next-themes` with ThemeProvider, OKLCH color system	☑ Working but needs audit
CMS/Admin	None	❌ Must build
Database	None	❌ Must add
Deployment	Not configured (Vercel-ready structure)	📝 Needs production config

2.2 Code Quality Observations

Strengths:

- Clean component architecture: `components/home/`, `components/product/`, `components/category/`, `components/ui/`

- Barrel exports (`components/home/index.ts`)
- Schema-driven product forms — this is a **strong pattern** that should be preserved and extended
- Proper path aliases (`@/*`)
- Theme-aware styling with CSS variables (OKLCH)
- `better-auth` and `react-hook-form` + `zod` already installed

Weaknesses / Technical Debt:

- **Dual Product interfaces** — `lib/products.ts` has `id: number`, `types/product.ts` has `id: number` but different shapes. `CartContext` has its own `CartItem` with `id: number`, while `types/product.ts` has `CartItem` with `id: string`. This is a ticking time bomb.
- **Hardcoded product data** — `lib/products.ts` contains 25 products with fabricated data (mock images, mock prices)
- **No catch-all routes** — Each category has its own directory (`/digital-paper-prints`, `/signage-solutions`, etc.) instead of `/categories/[slug]`
- **935-line ProductDetailPage.tsx** — Needs decomposition into smaller components
- **`createProductFromName`** — Dynamically fabricates products from slugs (for the all-products page). This is a clever hack but must be replaced by DB queries.
- **No API routes** — Zero server-side data flow
- **TanStack React Query** installed but hooks fully commented out
- **No error boundaries**
- **`ignoreBuildErrors: true`** in `next.config.ts` — Must be removed for production

2.3 Product Data Gap

The current `lib/products.ts` has **25 products** with mock data. The client provided a **Product Catalog PDF** at `docs/client-data/Products list.pdf`. This PDF must be:

1. Extracted and parsed for product names, categories, prices, and option variants
2. Mapped to the existing product form schema system
3. Seeded into Supabase with proper relational structure
4. Product images mapped to Supabase Storage buckets

---

3. Backend Strategy Analysis

3.1 Option A: Standalone Supabase (RECOMMENDED)

```
┌──────────────────────────────────────────────────────────────┐
│                    PRINTONLINE.ET v2.0                       │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌──────────────────────────────────────────────────────┐   │
│  │              NEXT.JS 16 FRONTEND                     │   │
│  │  App Router • React 19 • TanStack Query • Zod        │   │
│  └──────────────────────────┬───────────────────────────┘   │
│                             │                               │
│            ┌────────────────┼────────────────┐              │
│            │                │                │              │
```

```
│            ▼               ▼               ▼              │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐  │
│   │  API Routes  │  │ better-auth  │  │   Supabase       │  │
│   │  (Next.js)   │  │  (Auth)      │  │   Client (SDK)   │  │
│   └──────┬───────┘  └──────┬───────┘  └──────────┬───────┘  │
│          │                 │                     │          │
│          └─────────────────┴─────────────────────┘          │
│                            │                                │
│                            ▼                                │
│           ┌──────────────────────────────┐                  │
│           │     SUPABASE (PostgreSQL)    │                  │
│           │  • Products + Variants       │                  │
│           │  • Categories                │                  │
│           │  • Orders + Order Items      │                  │
│           │  • Customers (via Auth)      │                  │
│           │  • Product Images (Storage)  │                  │
│           │  • Design File Uploads       │                  │
│           │  • Row Level Security (RLS)  │                  │
│           └──────────────────────────────┘                  │
│                                                             │
│           ┌──────────────────────────────┐                  │
│           │    FUTURE: ERP INTEGRATION   │                  │
│           │  • Sync orders → Pana ERP    │                  │
│           │  • Sync inventory levels     │                  │
│           │  • Sync customer data        │                  │
│           └──────────────────────────────┘                  │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

Why Supabase:

- **Instant PostgreSQL** — No server provisioning, free tier sufficient for MVP
- **Auto-generated REST API** — Immediate CRUD without writing route handlers
- **Storage Buckets** — Product images and design file uploads out of the box
- **Row Level Security** — Fine-grained access control at DB level
- **Type Generation** — `supabase gen types typescript` generates TypeScript types from your schema (aligns with your schema-first philosophy from pana-erp)
- **Realtime** — Future: live order status updates
- **Edge Functions** — Future: serverless business logic if needed
- **Dashboard** — Minimalist CMS out of the box via Supabase Studio (temporary admin until custom CMS is built)

Schema-First Alignment with Your ERP Patterns:

Your Pana ERP follows `Frappe DocType → generate-types → TypeScript + Zod`. For PrintOnline:

```
Supabase Schema (SQL) → supabase gen types → TypeScript interfaces
                                                        │
                                                        ▼
                                        Zod schemas (manual, from types)
```

```
                                                    │
                                                    ▼
                                        TanStack Query hooks (generic)
                                                    │
                                                    ▼
                                        UI Components
```

3.2 Option B: ERP-Embedded CMS Module

This would add a "Web App" module to the pana-erp repo:

```
pana-erp/
├── app/
│   └── webapp/                    # CMS Module
│       ├── products/              # Product management
│       ├── orders/                # Order management
│       ├── customers/             # Customer management
│       └── settings/              # Web app config
```

Why This Is Tempting:

- Reuse the factory pattern (`createListHandler`, `useFrappeList`, etc.)
- CMS UI matches ERP UI (same theme system, same component library)
- Single deployment, single codebase
- Products managed in ERP → published to web app

Why This Is Risky Right Now:

1. **Business workflow misalignment** — You explicitly said there are misalignments in the ERP business workflow. Adding a web app module on top of an unstable ERP workflow is compounding risk.
2. **Deployment coupling** — ERP and web app have different deployment cadences. The web app needs to ship TODAY. The ERP is still in development.
3. **Frappe as backend** — Frappe is designed for internal ERP workflows, not public-facing e-commerce. Authentication (Frappe users vs. public customers), rate limiting, public API exposure — all require extra work.
4. **Schema dependency** — If you change an ERP DocType, it could break the web app. This coupling is dangerous during rapid iteration.

3.3 The Hybrid Strategy (Best of Both Worlds)

> [!TIP] **Ship with Supabase standalone today. Design the schema to be ERP-compatible from day 1. Build a sync layer later.**

```
TODAY (MVP):     PrintOnline.et ↔ Supabase (standalone)
LATER (v2.5):   PrintOnline.et ↔ Supabase ↔ Pana ERP (sync layer)
FUTURE (v3.0):  PrintOnline.et ↔ Pana ERP (direct, Supabase as cache/read
replica)
```

The Supabase schema should mirror ERP DocType field names where possible (e.g., `customer_name`, `item_name`, etc.) to make future sync trivial.

---

4. Recommended Architecture

4.1 Project Structure (v2.0)

```
printonline-et/
├── app/
│   ├── (storefront)/              # Public storefront (catch-all group)
│   │   ├── layout.tsx             # Storefront layout (Header + Footer)
│   │   ├── page.tsx               # Home page
│   │   ├── products/
│   │   │   └── [slug]/
│   │   │       └── page.tsx       # Product detail (unified)
│   │   ├── categories/
│   │   │   └── [slug]/
│   │   │       └── page.tsx       # Category page (catch-all)
│   │   ├── search/
│   │   │   └── page.tsx           # Search results
│   │   ├── cart/
│   │   │   └── page.tsx           # Cart page
│   │   ├── checkout/
│   │   │   └── page.tsx           # Checkout flow
│   │   ├── order-summary/
│   │   │   └── page.tsx           # Order summary
│   │   ├── order-confirmation/
│   │   │   └── page.tsx           # Order confirmation
│   │   └── contact/
│   │       └── page.tsx           # Contact page
│   │
│   ├── (auth)/                    # Auth pages
│   │   ├── login/
│   │   │   └── page.tsx
│   │   ├── register/
│   │   │   └── page.tsx
│   │   └── forgot-password/
│   │       └── page.tsx
│   │
│   ├── (account)/                 # Authenticated user area
│   │   ├── layout.tsx             # Account layout
│   │   ├── account/
│   │   │   └── page.tsx           # Account dashboard
│   │   └── orders/
│   │       ├── page.tsx           # Order history
│   │       └── [id]/
│   │           └── page.tsx       # Order detail
│   │
│   ├── (cms)/                     # Admin CMS (future, protected)
│   │   ├── layout.tsx             # CMS layout (ERP-style sidebar)
```

```
│   │   ├── cms/
│   │   │   ├── products/          # Product management
│   │   │   ├── orders/            # Order management
│   │   │   ├── customers/         # Customer management
│   │   │   └── settings/          # CMS settings
│   │
│   ├── api/                       # API Routes
│   │   ├── auth/
│   │   │   └── [...all]/
│   │   │       └── route.ts       # better-auth catch-all
│   │   ├── products/
│   │   │   └── route.ts           # Product API
│   │   ├── orders/
│   │   │   └── route.ts           # Order API
│   │   ├── upload/
│   │   │   └── route.ts           # File upload API
│   │   └── send-order-email/
│   │       └── route.ts           # Email notification
│   │
│   ├── globals.css
│   ├── layout.tsx                 # Root layout
│   ├── LayoutClient.tsx           # Client providers
│   └── not-found.tsx              # 404 page
│
├── components/
│   ├── ui/                        # Primitive UI (shadcn/ui)
│   ├── shared/                    # Shared reusable components
│   │   ├── ProductCard.tsx        # Universal product card
│   │   ├── ProductGrid.tsx        # Product grid layout
│   │   ├── SearchBar.tsx          # Global search
│   │   ├── PriceDisplay.tsx       # Price with currency
│   │   ├── QuantitySelector.tsx   # Quantity +/- control
│   │   ├── FileUpload.tsx         # Design file upload
│   │   ├── OrderSummaryCard.tsx   # Order summary display
│   │   └── CategoryNav.tsx        # Category navigation dropdown
│   ├── home/                      # Home page sections
│   ├── product/                   # Product detail components
│   ├── category/                  # Category page components
│   ├── cart/                      # Cart components
│   ├── checkout/                  # Checkout flow components
│   ├── account/                   # Account management components
│   └── cms/                       # CMS admin components (future)
│
├── lib/
│   ├── supabase/
│   │   ├── client.ts              # Browser Supabase client
│   │   ├── server.ts              # Server Supabase client
│   │   ├── admin.ts               # Service role client (API routes)
│   │   └── middleware.ts          # Auth middleware helper
│   ├── auth.ts                    # better-auth config
│   ├── products.ts                # Product query helpers (replaces hardcoded
data)
│   ├── query-client.ts            # TanStack Query client
│   ├── utils.ts                   # Utilities
```

```
│   ├── email.ts                    # Email transport (future)
│   └── email-template.ts           # Email template (future)
│
├── hooks/
│   ├── data/                       # Data fetching hooks
│   │   ├── useProducts.ts          # Product queries
│   │   ├── useCategories.ts        # Category queries
│   │   ├── useOrders.ts            # Order queries
│   │   └── useSearch.ts            # Search hook
│   ├── domain/                     # Business logic hooks
│   │   ├── useProductValidation.ts
│   │   ├── useCartManager.ts       # Enhanced cart with DB sync
│   │   └── useCheckout.ts          # Checkout flow state
│   └── ui/                         # UI hooks
│       ├── useDebounce.ts
│       └── useMediaQuery.ts
│
├── types/
│   ├── database.ts                 # Generated from Supabase (supabase gen types)
│   ├── product.ts                  # Product domain types
│   ├── order.ts                    # Order domain types
│   ├── customer.ts                 # Customer domain types
│   └── cart.ts                     # Cart types
│
├── context/
│   └── CartContext.tsx              # Cart provider (enhanced)
│
├── supabase/
│   ├── migrations/                 # SQL migrations
│   │   ├── 001_categories.sql
│   │   ├── 002_products.sql
│   │   ├── 003_product_variants.sql
│   │   ├── 004_customers.sql
│   │   ├── 005_orders.sql
│   │   ├── 006_storage.sql
│   │   └── 007_rls_policies.sql
│   ├── seed/
│   │   └── products.sql            # Initial product data from catalog
│   └── config.toml                 # Supabase local config
│
└── docs/
    ├── ARCHITECTURE_V2.md          # This document (finalized)
    ├── client-data/
    │   └── Products list.pdf       # Client product catalog
    └── pana-erp-docs/              # ERP reference docs
```

4.2 Route Group Strategy (Catch-All Directories)

The `(storefront)`, `(auth)`, `(account)`, and `(cms)` are **Next.js route groups**. They don't affect the URL but allow:

- Different layouts per group (storefront has Header/Footer, CMS has sidebar)

- Middleware-based auth checks per group
- Clean code organization

Key Change: Category pages move from `/digital-paper-prints` → `/categories/digital-paper-prints`. This eliminates 6 duplicate page files and uses one catch-all `categories/[slug]/page.tsx`.

---

5. Schema-First Domain Model

5.1 Core Database Schema (PostgreSQL via Supabase)

```
-- 001_categories.sql
CREATE TABLE categories (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  name TEXT NOT NULL UNIQUE,
  slug TEXT NOT NULL UNIQUE,
  description TEXT,
  image_url TEXT,
  display_order INTEGER DEFAULT 0,
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 002_products.sql
CREATE TABLE products (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  name TEXT NOT NULL,
  slug TEXT NOT NULL UNIQUE,
  category_id UUID REFERENCES categories(id) ON DELETE SET NULL,
  description TEXT,
  base_price DECIMAL(12,2) NOT NULL DEFAULT 0,
  sku TEXT UNIQUE,
  badge TEXT,                           -- 'Best Seller', 'New', 'Premium', etc.
  is_active BOOLEAN DEFAULT true,
  in_stock BOOLEAN DEFAULT true,

  -- SEO
  meta_title TEXT,
  meta_description TEXT,

  -- Product form type (maps to schema-driven form system)
  form_type TEXT NOT NULL DEFAULT 'paper',  -- 'paper', 'large-format', 'apparel',
'gift', 'board'

  -- JSON fields for flexible data
  features JSONB DEFAULT '[]'::jsonb,       -- ["Premium Quality", "Fast
Delivery"]
  specifications JSONB DEFAULT '[]'::jsonb, -- [{label: "Material", value:
"350gsm"}]

  created_at TIMESTAMPTZ DEFAULT NOW(),
```

```sql
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 003_product_images.sql
CREATE TABLE product_images (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  product_id UUID REFERENCES products(id) ON DELETE CASCADE,
  image_url TEXT NOT NULL,              -- Supabase Storage URL
  alt_text TEXT,
  display_order INTEGER DEFAULT 0,
  is_primary BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Product options (form fields with their available choices)
CREATE TABLE product_options (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  product_id UUID REFERENCES products(id) ON DELETE CASCADE,
  option_key TEXT NOT NULL,             -- 'size', 'paper', 'quantity', 'color'
  option_label TEXT NOT NULL,           -- 'Size', 'Paper Stock', 'Quantity'
  field_type TEXT NOT NULL DEFAULT 'select',  -- 'select', 'radio', 'checkbox',
'multi-select'
  is_required BOOLEAN DEFAULT true,
  display_order INTEGER DEFAULT 0,
  description TEXT,

  UNIQUE(product_id, option_key)
);

-- Option values (the selectable choices for each option)
CREATE TABLE product_option_values (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  option_id UUID REFERENCES product_options(id) ON DELETE CASCADE,
  value TEXT NOT NULL,                  -- 'a4', '14pt-gloss'
  label TEXT NOT NULL,                  -- 'A4 (210mm x 297mm)', '14 pt. Gloss'
  price_modifier DECIMAL(12,2),         -- Additional cost
  price_modifier_type TEXT DEFAULT 'fixed',  -- 'fixed', 'percentage',
'multiplier'
  group_name TEXT,                      -- For grouped selects
  description TEXT,
  display_order INTEGER DEFAULT 0,
  is_active BOOLEAN DEFAULT true
);

-- 004_customers.sql
-- (managed by better-auth, extended with profile)
CREATE TABLE customer_profiles (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id TEXT NOT NULL UNIQUE,         -- better-auth user ID
  full_name TEXT NOT NULL,
  email TEXT NOT NULL,
  phone TEXT,
  tin_number TEXT,                       -- Ethiopian TIN (Tax Identification
Number)
```

```sql
  company_name TEXT,

  -- Default delivery address
  address_line1 TEXT,
  address_line2 TEXT,
  city TEXT DEFAULT 'Addis Ababa',
  sub_city TEXT,
  woreda TEXT,
  country TEXT DEFAULT 'Ethiopia',

  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 005_orders.sql
CREATE TABLE orders (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  order_number TEXT NOT NULL UNIQUE,     -- POL-2026-00001 format
  customer_id UUID REFERENCES customer_profiles(id),

  -- Status workflow
  status TEXT NOT NULL DEFAULT 'pending',
  -- pending → confirmed → processing → ready → delivered → completed
  -- pending → cancelled

  -- Financials
  subtotal DECIMAL(12,2) NOT NULL DEFAULT 0,
  tax_amount DECIMAL(12,2) DEFAULT 0,
  delivery_fee DECIMAL(12,2) DEFAULT 0,
  total_amount DECIMAL(12,2) NOT NULL DEFAULT 0,

  -- Customer info (snapshot at order time)
  customer_name TEXT NOT NULL,
  customer_email TEXT NOT NULL,
  customer_phone TEXT,
  customer_tin TEXT,

  -- Delivery
  delivery_address TEXT,
  delivery_city TEXT,
  delivery_sub_city TEXT,

  -- Notes
  special_instructions TEXT,
  internal_notes TEXT,

  -- Terms
  terms_accepted BOOLEAN DEFAULT false,
  terms_accepted_at TIMESTAMPTZ,

  -- Email
  confirmation_email_sent BOOLEAN DEFAULT false,

  created_at TIMESTAMPTZ DEFAULT NOW(),
```

```sql
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE order_items (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  order_id UUID REFERENCES orders(id) ON DELETE CASCADE,
  product_id UUID REFERENCES products(id),

  -- Snapshot (prices at time of order)
  product_name TEXT NOT NULL,
  product_image TEXT,
  category TEXT,
  unit_price DECIMAL(12,2) NOT NULL,
  quantity INTEGER NOT NULL DEFAULT 1,
  line_total DECIMAL(12,2) NOT NULL,

  -- Selected options (snapshot)
  selected_options JSONB DEFAULT '{}'::jsonb,
  -- e.g., {"size": "A4", "paper": "14pt-gloss", "quantity": "500",
"productionTime": "rush"}

  -- Design file
  design_file_url TEXT,
  design_file_name TEXT,
  design_file_size INTEGER,

  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 006: Supabase Storage buckets (configured via dashboard or CLI)
-- Bucket: product-images (public)
-- Bucket: design-uploads (private, authenticated only)

-- 007_rls_policies.sql
-- Products: public read, admin write
-- Orders: user reads own, admin reads all
-- Customer profiles: user reads/writes own, admin reads all
```

5.2 Generated TypeScript Types

After running `supabase gen types typescript --local > types/database.ts`:

```typescript
// types/database.ts (auto-generated, DO NOT EDIT)
export type Database = {
  public: {
    Tables: {
      products: {
        Row: { id: string; name: string; slug: string; ... }
        Insert: { name: string; slug: string; ... }
        Update: Partial<{ name: string; slug: string; ... }>
      }
```

```
      // ... all tables
    }
  }
}

// Convenience types
export type Product = Database['public']['Tables']['products']['Row']
export type Category = Database['public']['Tables']['categories']['Row']
export type Order = Database['public']['Tables']['orders']['Row']
```

5.3 Product Image Strategy

```
Supabase Storage
├── product-images/          (Public bucket)
│   ├── business-cards/
│   │   ├── hero.jpg
│   │   ├── detail-1.jpg
│   │   └── detail-2.jpg
│   ├── led-shop-signs/
│   │   ├── hero.jpg
│   │   └── detail-1.jpg
│   └── ...
│
└── design-uploads/          (Private bucket, auth required)
    ├── {order_id}/
    │   └── customer-design.pdf
    └── ...
```

The `product_images` table stores references to Supabase Storage URLs. Images are organized by product slug in the bucket for easy management.

---

6. Feature-by-Feature Implementation Analysis

6.1 Product Catalog Data Migration

Step	Action	Detail
1	Extract PDF data	Parse `Products list.pdf` for names, categories, prices, options
2	Map to schema	Structure into `products`, `product_options`, `product_option_values`
3	Create seed SQL	`supabase/seed/products.sql`
4	Run migration	`supabase db push`
5	Map images	Upload to Storage, link in `product_images` table
6	Generate types	`supabase gen types typescript`

Step	Action	Detail
7	Replace `lib/products.ts`	Replace hardcoded data with Supabase queries

> [!WARNING] **Critical Path:** The product catalog PDF is the single most important input. We need to extract it FIRST before writing any backend code. The entire schema depends on understanding what options and variants exist per product.

6.2 Product Form Options (Schema-Driven)

The existing `ProductFormSchemas.ts` system is excellent. The v2.0 approach:

Current (v1): Form options are hardcoded in TypeScript files.
Future (v2): Form options are stored in DB (`product_options` + `product_option_values`) and fetched per product.

Migration Strategy: For MVP-today, keep the TypeScript schemas but feed them from DB data. The form rendering system stays the same.

```
// hooks/data/useProductOptions.ts
export function useProductOptions(productId: string) {
  return useQuery({
    queryKey: ['product-options', productId],
    queryFn: () => supabase
      .from('product_options')
      .select('*, product_option_values(*)')
      .eq('product_id', productId)
      .order('display_order'),
  });
}
```

6.3 Shopping Cart (Enhanced)

State	Behavior
Guest	Cart in localStorage (current behavior)
Authenticated	Cart synced to Supabase, survives across devices
Login	Merge localStorage cart with DB cart
Logout	Cart remains in localStorage

Implementation:

- Keep `CartContext` as-is for guest users
- Add DB sync layer when user authenticates
- Cart items must include selected options (mandatory form fields)

6.4 Add to Cart (Product Detail Only)

Per your requirement, the "Add to Cart" button only appears on product detail pages because options are mandatory. The current `ProductDetailPage.tsx` handles this with `handleProceedToOrder`. We need to:

1. Add to cart instead of navigate directly to order summary
2. Validate all mandatory form fields before allowing add-to-cart
3. Show cart count in header

6.5 Upload Design Feature

```
// components/shared/FileUpload.tsx
// - Accept: PDF, AI, PSD, PNG, JPG, SVG
// - Max size: 50MB (configurable)
// - Upload to Supabase Storage (design-uploads bucket)
// - Show progress bar
// - Return file URL for order attachment
```

The existing `ProductDetailPage.tsx` already has a `handleFileUpload` function (lines 115-121). This needs to be connected to Supabase Storage instead of just tracking state.

6.6 Order Summary & Customer Onboarding

Current flow: Product → Options → Order Summary (manual form) → Confirmation
New flow: Product → Options → Add to Cart → Cart → Checkout (auth gate) → Order Summary → T&C → Confirmation

```
Product Detail
    │
    │ [Add to Cart] (validates all mandatory options)
    ▼
Cart Page
    │
    │ [Proceed to Checkout]
    ▼
Auth Gate (login/register if not authenticated)
    │
    ▼
Order Summary (premium preview)
    │ • Product details + selected options
    │ • Customer info (from profile)
    │ • Special instructions text area
    │ • Delivery address
    │ • Design file upload(s)
    │ • Price breakdown
    │
    │ [✓ Terms & Conditions checkbox]
    │ [Place Order]
    ▼
Order Confirmation
    │ • Order number
```

```
    │ • Email confirmation sent
    │ • Status tracking
    ▼
User Account → Order History
```

6.7 Terms & Conditions

- Displayed as a modal/expandable section on the final ordering step
- Checkbox must be checked before "Place Order" is enabled
- `terms_accepted` + `terms_accepted_at` stored in order record
- T&C content stored as a markdown file or CMS-managed content

6.8 Special Instructions

- Free-text textarea on order summary page
- Stored in `orders.special_instructions`
- Displayed in order confirmation email

6.9 Global Search

```
// Implementation options:
// 1. Supabase Full-Text Search (PostgreSQL tsvector)
// 2. Supabase pg_trgm extension (fuzzy matching)
// 3. Client-side filtering (for MVP with small catalog)

// Recommended: PostgreSQL full-text search
CREATE INDEX products_search_idx ON products
  USING gin(to_tsvector('english', name || ' ' || coalesce(description, '')));

// Hook
export function useSearch(query: string) {
  return useQuery({
    queryKey: ['search', query],
    queryFn: () => supabase
      .from('products')
      .select('id, name, slug, base_price, category:categories(name)')
      .textSearch('name', query, { type: 'websearch' }),
    enabled: query.length > 2,
  });
}
```

6.10 Navigation Dropdown (Category Mega-Menu)

```
// components/shared/CategoryNav.tsx
// - Fetches categories with product counts from DB
// - Shows products under each category on hover
// - Premium dropdown with product thumbnails
// - Uses @radix-ui/react-navigation-menu (already installed)
```

6.11 Email Confirmation

Already documented in `/email-integration` workflow. Implementation plan:

1. Install `nodemailer`
2. Create SMTP transport with Ethio Telecom config
3. Send order confirmation to `order@printonline.et`
4. Send order confirmation to customer email
5. Fire-and-forget (order succeeds even if email fails)

6.12 User Account Management

Feature	Priority
Profile (name, email, phone, TIN)	MVP
Order history	MVP
Order status tracking	MVP
Saved addresses	Post-MVP
Re-order from history	Future
Wishlist	Future

---

7. Authentication Strategy: better-auth + Supabase

7.1 Why better-auth over Supabase Auth?

Factor	better-auth	Supabase Auth
Already installed	☑ In package.json	Would need additional setup
Session management	Server-side, secure	JWT-based
DB adapter	Supports Supabase/PostgreSQL	Native
Customization	Highly flexible	Opinionated
ERP compatibility	Can sync to any backend	Supabase-only
Email/Password	Built-in	Built-in
Social auth	Plugin-based	Built-in

Recommendation: Use **better-auth** with **Supabase as the database adapter**. This gives you:

- Full control over auth flow
- Custom user fields (TIN, company_name)
- Easy ERP sync in the future
- Server-side session management

7.2 Auth Flow

```
Register → better-auth creates user → customer_profiles row created (DB trigger or
API)
Login → Session created → Cart synced from localStorage to DB
Checkout → Auth required → Redirect to login if not authenticated
```

7.3 TIN Field Integration

The Ethiopian Tax Identification Number (TIN) is optional during registration. It's stored in `customer_profiles.tin_number` and displayed on order confirmations/invoices.

---

8. Frontend Architecture Modernization

8.1 Component Unification Strategy

Problem: Duplicate/similar components across pages.
Solution: Create `components/shared/` with unified, reusable components.

Component	Used In	Notes
`ProductCard`	Home, Category, Search, All Products	Single card component with grid/list variants
`ProductGrid`	Category, Search, All Products	Grid layout with filtering
`PriceDisplay`	Product card, detail, cart, checkout	Consistent price formatting with ETB
`SearchBar`	Header (global)	Debounced search with dropdown results
`QuantitySelector`	Product detail, cart	+/- buttons with input
`FileUpload`	Product detail, order summary	Drag-and-drop with progress
`OrderSummaryCard`	Order summary, confirmation	Order item display
`CategoryNav`	Header	Mega-menu dropdown

8.2 Dark Mode Audit

The current theme system uses OKLCH colors with Pana branding. The `next-themes` ThemeProvider defaults to `light`. Key audit points:

- ☑ CSS variables properly defined for both themes
- ⚠ Need to check all `bg-white`, `text-gray-*`, `border-gray-*` instances in components
- ⚠ ProductDetailPage.tsx (935 lines) likely has hardcoded colors
- ⚠ Home page sections need verification
- ⚠ Checkout/order pages need verification

8.3 Catch-All Routes

Current (6 separate category pages):

```
/digital-paper-prints/page.tsx
/signage-solutions/page.tsx
/flex-banners/page.tsx
/vinyl-prints/page.tsx
/fabric-prints/page.tsx
/promotional-items/page.tsx
```

New (1 catch-all):

```
/categories/[slug]/page.tsx  → Handles ALL categories
```

The existing `CategoryTemplate` component already supports this pattern. We just need to wire it to DB queries instead of hardcoded data.

8.4 ProductDetailPage Decomposition

The 935-line `ProductDetailPage.tsx` should be split:

```
components/product/
├── ProductDetailPage.tsx      # Main orchestrator (200 lines max)
├── ProductGallery.tsx         # Image gallery with zoom
├── ProductInfo.tsx            # Name, price, badge, rating
├── ProductForm.tsx            # Options form (uses schema system)
├── ProductTabs.tsx            # Description, specs, reviews tabs
├── ProductActions.tsx         # Add to cart, wishlist, share
├── ProductBreadcrumb.tsx      # Breadcrumb navigation
└── RelatedProducts.tsx        # Related products section
```

> [!NOTE] **Deferrable:** Per your instruction to "mostly defer UI tasks," the decomposition can happen incrementally. The priority is getting the backend wired up with the existing UI.

---

9. MVP-Today Scope vs Full v2.0 Roadmap

9.1 MVP-Today (Ship This Afternoon) ▨

#	Task	Time Est.	Dependencies
1	**Create Supabase project**	10 min	None
2	**Write & run SQL migrations** (categories, products, options)	30 min	Supabase project
3	**Extract product catalog PDF** → seed data	45 min	PDF analysis
4	**Generate TypeScript types** from Supabase	5 min	Migrations

#	Task	Time Est.	Dependencies
5	**Create Supabase client libs** (browser + server)	15 min	Supabase project
6	**Replace** `lib/products.ts` with Supabase queries	30 min	Client libs + types
7	**Upload product images** to Supabase Storage	30 min	Storage bucket
8	**Update ProductDetailPage** to fetch from DB	30 min	Query hooks
9	**Update category pages** to fetch from DB	20 min	Query hooks
10	**Environment variables** + deploy to Vercel	15 min	All above

MVP-Today Total: ~4 hours

9.2 Sprint 1 (This Week) ◍

#	Task	Priority
1	better-auth setup + login/register pages	●
2	Customer profile with TIN	●
3	Cart DB sync for authenticated users	●
4	Order creation + persistence to DB	●
5	Order confirmation email (Nodemailer)	●
6	Terms & conditions on checkout	●
7	Special instructions field	◍
8	Design file upload to Supabase Storage	◍

9.3 Sprint 2 (Next Week) ◍

#	Task	Priority
1	Global search (PostgreSQL full-text)	●
2	Account dashboard + order history	●
3	Category mega-menu dropdown	◍
4	Catch-all category routes	◍
5	Product form options from DB	◍
6	Dark mode audit & fixes	◍
7	Component unification (shared/)	◍

9.4 v2.0 Full Release (2-3 Weeks)

#	Task	Priority

#	Task	Priority
1	CMS admin panel (product CRUD)	●
2	Order management CMS	●
3	Customer management CMS	○
4	Payment gateway integration (Telebirr, CBE Birr)	○
5	ProductDetailPage decomposition	○
6	Performance optimization (ISR, caching)	○
7	SEO optimization (structured data, sitemaps)	○

9.5 v2.5+ (Deferred)

#	Task
1	Pana ERP integration (sync layer)
2	International reach (multi-currency, shipping)
3	Inventory sync from ERP
4	WhatsApp Business integration
5	Payment reconciliation
6	Analytics dashboard

---

10. Risk Assessment & Pitfalls

10.1 Technical Risks

Risk	Severity	Mitigation
Product catalog PDF parsing errors	● High	Manual verification of extracted data before seeding
Supabase free tier limits	○ Medium	500MB DB, 1GB Storage, 2GB bandwidth. Monitor usage, upgrade when needed
better-auth + Supabase compatibility	○ Medium	Use Drizzle adapter for better-auth, test early
Product image mapping	○ Medium	Current 30 sample images are reused. Need real product photos from client
Ethio Telecom SMTP reliability	○ Medium	Self-signed certs, potential blocked ports. Test from Vercel, add retry logic
Type drift between DB and frontend	○ Low	`supabase gen types` + CI check

Risk	Severity	Mitigation
Dark mode regression	Low	Audit after backend migration, not a blocker
935-line component	Low	Works, decompose incrementally

10.2 Business Risks

Risk	Severity	Mitigation
No real product images	High	Client must provide. Use current samples as placeholders with clear labeling
Pricing accuracy	High	Verify all prices from PDF against client confirmation
No payment processing	Medium	MVP accepts orders without payment. Manual payment collection by Pana Promotion staff
No inventory management	Medium	All products show "In Stock" for MVP. Manual management via CMS
Legal: T&C content	Medium	Client must provide T&C text. Use placeholder for MVP

10.3 Anti-Patterns to Avoid

Anti-Pattern	Why It's Bad	Do This Instead
Keeping hardcoded products alongside DB	Data inconsistency	Remove `lib/products.ts` after DB migration
Using Supabase client directly in components	No caching, no loading states	Use TanStack Query hooks
Skipping RLS policies	Security vulnerability	Set up RLS on day 1
Storing passwords/secrets in repo	Security breach	`.env.local` only
Building CMS before storefront works	Wrong priority	Supabase Studio as temporary CMS
Over-engineering for international before Ethiopia works	Feature creep	ETB only, Ethiopia addresses only

11. Follow-Up Questions for Decision

Before we execute, I need your input on these critical decisions:

11.1 Backend & Infrastructure

1. **Supabase Account:** Do you have a Supabase account? Should I set up the project config for you?
2. **Supabase Region:** Which region? The closest to Ethiopia would be `eu-central-1` (Frankfurt) or `ap-south-1` (Mumbai). Frankfurt is typically lower latency.

3. **better-auth vs Supabase Auth:** I recommended better-auth since it's already in your `package.json`. Are you aligned with this, or would you prefer Supabase Auth for simplicity?

11.2 Product Data

4. **Product Catalog PDF:** Can I attempt to read/extract the PDF data now? This is the critical path for the MVP.
5. **Product Prices:** Are the prices in the PDF in **ETB (Ethiopian Birr)** or USD? The current mock data uses USD-looking prices.
6. **Product Images:** Do you have real product images from the client, or should we use the existing 30 sample images for MVP?

11.3 Feature Scope

7. **Payment Processing:** For MVP, should we just capture orders (no payment processing), or do you have a payment gateway ready (Telebirr, CBE Birr)?
8. **Terms & Conditions:** Do you have the T&C text from the client, or should we use a placeholder?
9. **Cart "Add to Cart" only on detail page:** Confirmed. The floating cart icon in the header should show count — should it navigate to `/cart` or open a slide-over drawer?

11.4 CMS Strategy

10. **Temporary CMS:** For the MVP-today push, are you comfortable using **Supabase Studio** (the built-in dashboard) as a temporary CMS to manage products/orders? This lets us skip building a custom CMS and focus on the storefront.
11. **Future CMS:** When we build the custom CMS, should it live at `/cms/...` in this repo, or as a separate deployment?

11.5 Deployment

12. **Hosting:** Vercel (most natural for Next.js)? Or do you have a different preference?
13. **Domain:** Is `printonline.et` already pointing to a Vercel deployment, or needs DNS setup?
14. **Environment:** Do you have `.env.local` set up with any credentials (SMTP, etc.)?

11.6 ERP Integration Timeline

15. **ERP Sync:** When do you estimate the Pana ERP business workflow will be stable enough for integration? This helps us design the Supabase schema to be ERP-compatible from day 1.

---

12. Technology Stack Summary

Layer	Technology	Version	Purpose
Framework	Next.js	16.x (App Router)	Full-stack React framework
Language	TypeScript	5.x (Strict)	Type safety
UI Library	React	19.x	Component framework
Styling	Tailwind CSS	v4.x	Utility-first CSS

Layer	Technology	Version	Purpose
Component Library	shadcn/ui	Latest (new-york)	Primitive UI components
State Management	TanStack Query	v5.x	Server state + caching
Forms	React Hook Form	v7.x	Form handling
Validation	Zod	v3.x	Runtime type validation
Animation	Framer Motion	v11.x	Micro-interactions
Icons	Lucide React	Latest	Iconography
Toasts	Sonner	Latest	Notifications
Theme	next-themes	v0.4.x	Dark/light mode
Database	Supabase (PostgreSQL)	Latest	Primary data store
File Storage	Supabase Storage	Latest	Images + design uploads
Auth	better-auth	v1.3.x	Authentication
Email	Nodemailer	Latest	SMTP notifications
Deployment	Vercel	Latest	Hosting + CDN

> [!CAUTION] **This document is a PRE-DECISION analysis. No code changes should be made until you've reviewed and confirmed the architectural decisions above, especially the backend strategy and follow-up questions in Section 11.**

Awaiting your review and decisions, Kidus. Once confirmed, we'll begin execution with the schema-first approach: migrations → types → seed data → queries → UI wiring.